

Section 3 – Hands-On: Prompt Engineering for Sigma Queries

1. Goal of This Part

Learn to use Large Language Models (LLMs) like **Gemini**, **ChatGPT**, and **Claude** to:

- Generate **Sigma rules** directly from natural-language threat intelligence
- Validate, refine, and correct LLM outputs for **schema**, **semantic**, and **context** accuracy
- Apply **prompt-engineering techniques** for deterministic, high-precision rule generation

2. Core Principles of Prompt Design

Principle	Why It Matters	Example
Clarity	Ambiguous prompts → vague or hallucinated rules	“Generate a Sigma rule detecting <i>PowerShell EncodedCommand</i> execution on Windows process creation logs.”
Context	Tells model the role, goal, and target schema	“You are a SOC detection engineer writing Sigma rules for Elastic SIEM.”
Constraint	Prevents drift / schema violations	“Use only fields from the Sigma process_creation schema.”
Determinism	Low randomness = stable YAML output	temperature = 0.2–0.3
Validation	Ask model to self-check its rule	“At the end, verify that the condition matches the threat intent.”

3. Prompt Template (General)

You are a senior SOC detection engineer.

Task: Generate a Sigma rule for detecting <THREAT/TECHNIQUE>.

Platform: <Elastic SIEM | Sentinel | Splunk>.

Constraints:

- Follow Sigma v2 schema.
- Use fields from <process_creation | dns | network_connection>.

- Include metadata: title, id, status, logsource, detection, condition, level.
- Add one benign exclusion to reduce false positives.

Output: YAML Sigma rule only, no extra text.

After generation, explain in one sentence how the rule aligns with MITRE ATT&CK T#

Example filling:

<THREAT/TECHNIQUE> = PowerShell EncodedCommand Execution (ATT&CK T1059.001)

<Platform> = Elastic SIEM

4. Good Prompt vs Weak Prompt Comparison

Weak Prompt	Result	Improved Prompt	Result
“Write a Sigma rule for PowerShell.”	Too generic → matches all PowerShell runs	“Generate a Sigma rule detecting PowerShell EncodedCommand usage (T1059.001) on Windows process_creation logs; include an exclusion for legitimate admin scripts.”	Specific rule targeting malicious patterns

5. Example Prompt and Output

Prompt (ready to use):

You are a SOC detection engineer.

Generate a Sigma rule for Elastic SIEM detecting PowerShell EncodedCommand execution (MITRE T1059.001).

Use logsource=process_creation, product=windows.

Add filter for known benign admin scripts.

Return only valid Sigma YAML.

Typical LLM Output (Expected):

title: PowerShell EncodedCommand Execution

id: auto-T1059-001

status: experimental

logsource:

product: windows

category: process_creation

detection:

selection:

Image|endswith: '\powershell.exe'

CommandLine|contains: '-EncodedCommand'

filter:

CommandLine|contains: 'AdminScript.ps1'

condition: selection and not filter

level: high

tags:

- attack.execution

- attack.t1059.001

6. LLM-Specific Prompting Tips

Gemini (Google)

- Very structured but sometimes truncates YAML.
- Add: *“Return complete Sigma rule, even if long.”*
- Supports citations → great for rule rationale.
- Example addition:

“Include one-line rationale and MITRE reference at bottom.”

ChatGPT (OpenAI GPT-4)

- Excellent at MITRE mapping and YAML format.
- Sometimes hallucinates metadata (fake IDs).
- Add: *“Do not invent arbitrary UUIDs; leave id as auto-Txxxx.”*

Claude (Anthropic)

- Keeps context very well, good for chain-of-thought reasoning.
- May summarize instead of output YAML → add:

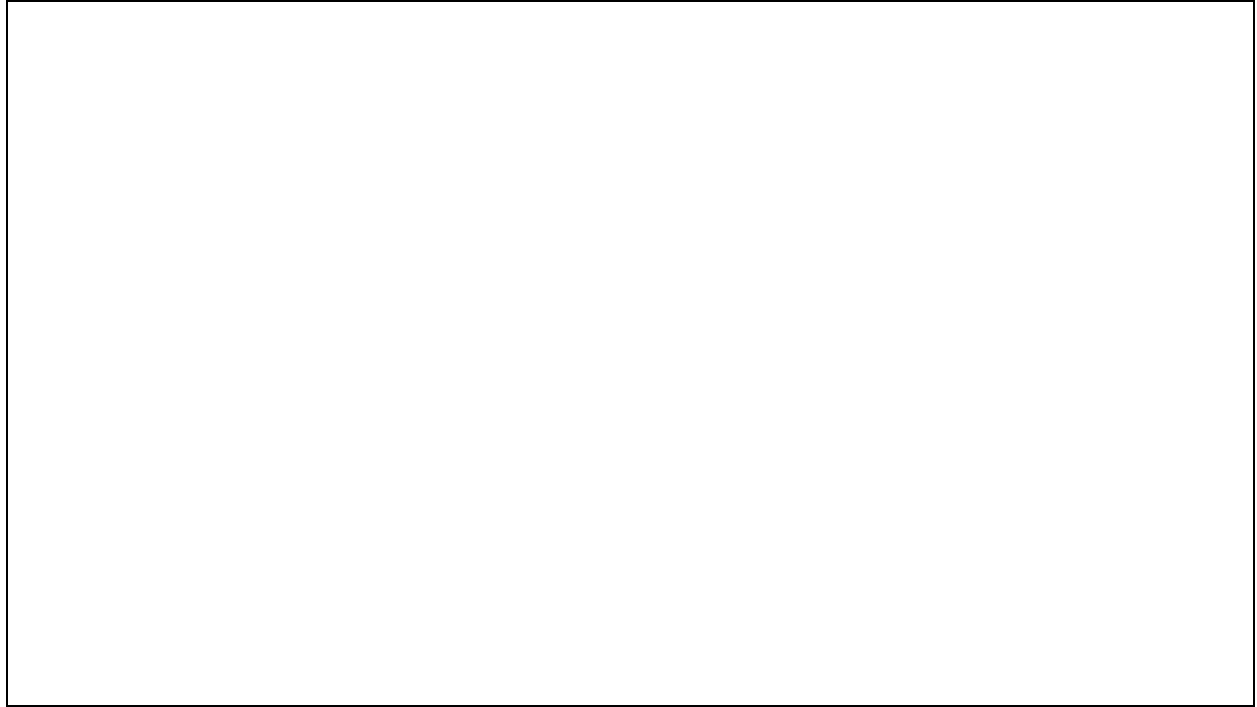
“Output only Sigma YAML, no explanation.”

7. Few-Shot Prompting (Consistency Booster)

Below a valid Sigma rules. Follow their structure.

<Rule 1: PowerShell EncodedCommand>

Now write a rule detecting MSHTA launch with HTTP payloads (ATT&CK T1218.005).



8. Chain-of-Thought Prompting (Self-Validation)

Ask model to reason first → then generate → then verify.

Step 1: List log fields and patterns that indicate this technique.

Step 2: Write a Sigma rule detecting them.

Step 3: Validate the rule by explaining how it avoids false positives.

Output: reasoning + YAML rule.

- Produces better structured logic, easier to audit.
- Longer response time; use only for complex detections.

9. Validation Checklist (LLM Output QA)

Check	Tool / Method	Pass Criteria
Syntax	yaml lint, sigmac --validate	YAML parse OK
Schema	Compare fields to Sigma spec	Only valid field names
Semantics	Review vs MITRE TTP	Rule detects intended action
Precision	Test on baseline logs	No benign hits
Completeness	All required keys present	title, status, logsource, detection, condition, level

10. Refinement Prompts (Quick Fixes)

Goal	Prompt
Add Exclusions	“Add filters to exclude Windows Update and legitimate admin scripts.”
Increase Precision	“Tighten this rule by matching only when -EncodedCommand appears with powershell.exe.”
Cross-Platform Conversion	“Convert this Sigma rule to a KQL query for Microsoft Sentinel.”
Explain the Rule	“Summarize this rule in one sentence with MITRE mapping.”
Check for Over-Coverage	“Does this rule risk detecting legitimate PowerShell administration? Suggest one filter.”

11. Multi-Model Ensemble Approach

1. Run the same prompt in Gemini, ChatGPT, and Claude.
2. Compare outputs:
 - field names used
 - presence of filter logic
 - compliance with Sigma schema
3. Choose best structure or merge elements → hybrid rule.
4. Store both prompt and result in Git for traceability.

12. Sample Exercise (Individual or Team)

Task: Create and validate a Sigma rule for “Certutil File Download (ATT&CK T1105).”

Steps

1. Copy prompt template from Section 3 above.
2. Run it on two different LLMs.
3. Use sigmac --validate or YAML lint tool to check syntax.
4. Manually review rule coverage vs expected behavior.
5. Modify prompt → reduce false positives.
6. Submit before/after comparison + rationale.

13. Common Failure Patterns and Fixes

Symptom	Root Cause	Fix
Missing logsource	LLM not instructed to include metadata	Add explicit constraint “include logsource section”
Wrong field names	Model invented fields	Embed schema in prompt or post-validate via script
Over-broad condition	LLM omitted specific arg matching	Add “focus on arguments indicating malicious behavior”
Output in Markdown	ChatGPT default format	Add “Return raw YAML only”
Hallucinated UUIDs	Generative metadata	Instruct “leave id as auto-T####”

14. Operational Integration Tips

- **Version-control** both prompt and rule (rule.yml, prompt.txt).
- Run nightly validation pipelines checking for schema drift.
- Link prompt → commit → SOC change ticket for traceability.
- Periodically review top LLM-generated rules for FP rates.

15. Quick Reference Prompts

Purpose	Copy-Ready Prompt
Generate Sigma rule	“Generate a Sigma rule for {Technique / TTP} using Elastic process_creation schema. Include metadata and MITRE tags.”
Validate existing rule	“Review this Sigma rule for field accuracy and false positives. Suggest one improvement.”
Convert to KQL	“Translate this Sigma YAML rule into a KQL query for Sentinel.”
Explain logic	“Summarize the detection intent and risk of this Sigma rule in two sentences.”
Auto-generate filters	“Add filters to reduce false positives from administrative tools.”

16. Wrap-Up and Next Steps

- LLMs can create Sigma rules quickly but require explicit prompts for accuracy.
- Use **few-shot**, **chain-of-thought**, and **guardrails** to stabilize outputs.
- Always validate and store prompts as part of the rule lifecycle.
- Combine multiple LLMs (Gemini + ChatGPT + Claude) for cross-verification.